Algorithm 1: MAPPO-Safe-EWC Training Procedure

```
Require: Network N, PV penetration levels P = {0%, 20%, ..., 100%}
Require: Curriculum stages S = {0, 1, ..., 5}, advancement threshold τ = 0.95
Require: Hyperparameters: γ, λ_GAE, ε_clip, lr_actor, lr_critic, λ_EWC

Initialize shared actor parameters θ, centralized critic φ
Initialize Lagrange multipliers λ_j ← 1.0 ∀j ∈ P/B pairs
Initialize penalty coefficient ρ ← 10.0
Initialize EWC task memory M ← ∅

for stage s = 0 to 5 do
    Set PV level p_s ∈ P and topology condition
    while CTI_satisfaction_rate < τ over 100 episodes do
        // Collect rollouts
        for t = 1 to T_rollout do
            for each agent i = 1 to N do
                o_i ← local_observation(V, I, P_dir, I_fault, ...)
                m_i ← Encoder_θ(o_i)                    // 10-dim → 8-float
                m̄_i ← Σ_j∈N_i α_ij · m_j                 // GNN attention
                õ_i ← [o_i ‖ m̄_i]                       // 18-dim enhanced obs
                a_i ← μ_θ(õ_i) + σ_θ · ε,  ε ~ N(0,I)   // Gaussian sample
                a_i^safe ← P(a_i)                       // Action projection
            end for
            s' ← T(s, a^safe, ξ)                        // Environment step
            r ← R(s, a^safe)                              // Team reward
            Store (o, õ, a^safe, r, o', V_φ(s), log π_θ) in buffer
        end for
        // PPO policy update
        Compute advantages Â_t using GAE(λ_GAE, γ)
        L_CLIP(θ) = E_t[min(r_t(θ)·Â_t, clip(r_t(θ), 1±ε)·Â_t)]

        // Safe RL constraint cost
        g_j ← 0.3 - (t_bkp,j - t_pri,j)  ∀j ∈ P/B pairs
        L_AL(θ) = -Σ_j λ_j·E[g_j] - (ρ/2)·Σ_j E[max(0, g_j)²]

        // EWC continual learning penalty
        if |M| > 0 then
            L_EWC(θ) = (λ_EWC/2)·Σ_task∈M Σ_i F_task,i·(θ_i - θ*_task,i)²
        end if

        // Combined gradient update
        θ ← θ - α·∇_θ[L_CLIP + L_AL + L_EWC]
        φ ← φ - α_critic·∇_φ[(V_φ(s) - R_GAE)²]

        // Dual ascent for Lagrange multipliers
        λ_j ← max(0, λ_j + α_λ·ḡ_j)  with PID stabilization

        // Adaptive penalty coefficient
        if violation_rate > 0.1 then ρ ← ρ × 1.5
        elif violation_rate < 0.05 then ρ ← ρ / 1.5
        end if
    end while

    // EWC task consolidation at stage transition
    F ← estimate_fisher(θ, D_current)      // Monte Carlo sampling
    M ← M ∪ {(task_s, θ, F, λ_EWC)}       // Store task memory

    if task_similarity(new_task, M) > 0.8 then
        θ ← warm_start_from_most_similar(M)
    end if
end for

return θ*, φ*, λ*, M
```